Devlup Official Website

Overview
This is a full-stack website for DevlUp Labs with a FastAPI backend and a React + Vite frontend.
The backend serves REST APIs for blogs, videos, podcasts, team members, timeline events, comments, and contact form submissions.
The frontend consumes those APIs and focuses heavily on visual presentation for both dark/light theme,animation-driven interaction with 3-D experience,responsive layouts,dynamic search/filter systems and centralised administrative workflow using admin system.

Backend

Tech Stack
- Python 3.12
- FastAPI
- MongoDB
- Motor / PyMongo
- Pydantic
- python-dotenv
- python-jose (JWT)
- passlib[bcrypt]
- cloudinary
- requests
- APScheduler

Architecture
- `backend/main.py` configures FastAPI, CORS, and route registration.
- `backend/database.py` connects to MongoDB via `MONGO_URL` and exposes both sync and async clients.
- `backend/routes/devlup/` contains resource-specific API routers.
- `backend/models/devlup/` defines Pydantic schemas for validation.
- `backend/dependencies/` contains auth dependencies and route guards, including the admin JWT validator.
- `backend/core/` contains core security helpers such as password hashing, verification, and JWT creation.
- `backend/services/` handles Cloudinary uploads for images and media.
- `backend/config/cloudinary.py` configures Cloudinary using environment variables.

Environment Variables
Required environment variables (used across backend and services):
- `MONGO_URL`

- `DB_NAME` (database name)
- `SECRET_KEY`
- `CLOUDINARY_CLOUD_NAME`
- `CLOUDINARY_API_KEY`
- `CLOUDINARY_API_SECRET`
- `ADMIN_EMAIL`
- `ADMIN_PASSWORD`

Backend Features

Authentication & Security
The backend uses JWT-based authentication for protected admin routes.
Current implementation includes:
- login using OAuth2 password flow
- JWT token creation and verification
- admin route protection using `admin_required`
- password hashing with bcrypt
The current auth system is intentionally simple and mainly focused
on admin-controlled workflows(verifies role==admin in JWT).
Possible future improvements:
- role-based access control/multi-admin permissions(if there are multiple roles)
- audit logging(if you want to store records of admin actions such as deletion,edits,etc.)
- refresh tokens(if you want to avoid repeatead login in any situation)

Blogs
The blogs system supports full CRUD operations with media support.
Current features:
- create, update, fetch, and delete blogs
- thumbnail and optional media uploads,optional external URLs
- tag-based organization
- lightweight preview responses for blog lists
- detailed single-blog retrieval
The structure is flexible and can later be expanded into a more CMS-like system.
Possible future improvements:
- pagination and search-if content grows larger-for better user navigation(maybe you can use CMS for this)
- SEO metadata support for better search-engine visibility(if needed- it will help you to improve discoverability.)
- scheduled publishing to automate blog publishing instead of manual publishing
- markdown editor support(if you want readymade formatting)
- draft/published states(if you want to save blogs as drafts before making it publicly visible)

Videos
The video system currently uses YouTube RSS feeds
instead of the official YouTube API.
Current features:
- RSS feed parsing
- automatic video metadata storage
- category management
- lightweight video ID endpoints
- video statistics
- RSS preview before database insertion
RSS was used to simplify setup and avoid API quota limitations.
Possible future improvements:
- YouTube Data API integration instead of RSS
- automatic scheduled syncing
- richer metadata support
- playlist/channel management

Podcasts
The podcast system supports uploaded or externally linked media.
Current features:
- podcast CRUD operations
- thumbnail uploads
- optional uploaded audio/video
- external media links
- tag support
The structure is intentionally flexible
to support different podcast hosting approaches.
Possible future improvements:
- transcript support(text transcripts for podcast episodes)
- streaming optimizations(adaptive streaming to avoid buffering,chunked media loading for low latency,lower bandwith usage,Cache stargies for temporary storage)
- analytics(if you want to track user engagement in podcast)
- podcast RSS generation(if you wish it to distribute it to other platforms)

Team
The team system supports both public profiles
and hidden/private member data.

Current features:
- public member profiles
- hidden member metadata
- secret-code-based hidden access
- hidden comments/contribution tracking
- admin-protected member management
- Cloudinary image uploads
The hidden-data system currently uses lightweight code-based access
instead of a full permission system.
Possible future improvements:
- Google Sheets/Form integration and no manual filling of data(can be done using CSV-just like XML it gives you CSV URL,Google-API or App Script(doesn't run on localhost) based syncing)
- portfolio auto-generation
- encrypted hidden data
- role-based visibility(if needed)

Timeline
The timeline system manages platform events and milestones.
Current features:
- timeline CRUD operations
- flexible event structure
- frontend visualization support
The current structure is intentionally lightweight
to support different UI presentation styles.
Possible future improvements:
- media-supported events(can add images,videos,external links)
- categorized timelines(to divide it into
events,sessions,achievements,releases,announcements,etc.)

Contact
The contact system stores visitor inquiries.
Current features:
- validated contact form submissions
- admin-side retrieval
- deletion workflows
The current implementation behaves like a lightweight inquiry-management system.
Possible future improvements:
- email automation(Automatic emails can be sent after a user submits
  a contact form)-If you wish

- Notification to co-ordinator/admin-if any query/request + keep status of wheter solved or not(pending/in progress/resolve)
- spam filtering(using CAPTCHA,blocked keywords,email validation)
- CRM integration-to enable automatic data sharing

Comments(it is used in blogs)
The comments system provides blog discussion support.
Current features:
- blog-specific comments
- admin moderation access
- comment deletion
- validation limits

The current implementation is intentionally simple.
Possible future improvements:
- threaded replies(Users can reply directly to other comments for further discussion)
- reactions/likes/upvotes/emojis
- moderation queue(Comments can first enter a review queue
  before becoming publicly visible to avoid spam/inappropriate)
- anti-spam systems-Additional protection systems can be added
  to reduce bot/spam submissions.

Admin System
The backend includes a centralized admin-management workflow.
Current features:
- protected admin routes
- content management workflows
- user-management utilities
- dashboard access protection

The current admin system is lightweight and focused on internal management.
Possible future improvements:
- analytics dashboard(Admins can view platform statistics and usage insights through a centralized dashboard.)
- multi-admin support
- granular permissions(different admin different level of access)
- activity logs(Store records of important admin/system actions.)

Media Upload System
Media uploads are currently handled using Cloudinary.
Current features:
- image uploads
- video/media uploads
- hosted media URLs
- CDN-based delivery
Cloudinary was selected to simplify media hosting and reduce infrastructure complexity.
Possible future improvements:
- media compression pipelines(Uploaded images/videos can automatically be optimized before storage or deliveryfor improving loading)
- AWS S3/Firebase storage
- self-hosted storage systems(instead of third-party)-but quite critical
---

EXtra:-
 - Add unit/integration tests for API routes.
 - Replace wildcard CORS origin with trusted domains in production.
 - Add backend health and metrics endpoints.

Frontend

Tech Stack
Core Framework & Build Tools
- React 19
- Vite
- React Router DOM v6
Styling & UI Libraries
- Tailwind CSS v4
- React Icons
- Lucide React
- Font Awesome
Animation & Motion
- GSAP
- Framer Motion
3D Rendering & Visual Effects
- React Three Fiber
- @react-three/drei
- three
- postprocessing
- @react-three/postprocessing
Networking & API Integration
- axios

Architecture
- `frontend/src/App.jsx` defines routes, theme context, and shared layout.
- `frontend/src/api/axios.js` creates an axios instance with base URL and auth token interceptor.
- `frontend/src/api/services.js` centralizes all backend API calls.
- `frontend/src/pages/` contains route-level wrappers for page components.
- `frontend/src/components/` contains reusable UI and feature components.
- `frontend/src/admin/components/` contains the admin panel UI and CRUD forms.

Routing
- Public routes:
  - `/` → Home
  - `/blog` → Blog list
  - `/blogs/:id` → Single blog view
  - `/team` → Team page
  - `/timeline` → Timeline page
  - `/video` → Video page
  - `/podcast` → Podcast page
  - `/portfolio/:username` → Portfolio page
- Admin routes:
  - `/login` → Login page
  - `/dashboard` → Admin dashboard (protected)
  - `/403` → Forbidden page

Shared Features
- Dark/light theme toggling with animated background changes.
- Global `ThemeContext` for layout state: theme, hamburger menu, search and filter toggles.
- Responsive behaviour via window resize listeners in several components.

Frontend Features by Component

Home
- 3D interactive hero built with React Three Fiber and custom shaders
- Animated 3D discs, floating blocks, environment lighting, and camera movement(3D canvas in home with graphical experience)
- Likely contains a sophisticated presentation section with custom 3D visuals

Blog
- Blog list page component with content fetched from backend.
- `BlogView` displays a full blog article with hero image, tags, metadata, and body text.
- Comments section on `BlogView` includes:
  - fetch comments for a blog
  - post a new comment
  - delete comments

Videos
- `frontend/src/components/Videos.jsx` loads video metadata via `getVideos()`.
- Supports search and tag filtering.
- Provides google-like suggestions as the user types.
- Renders `Cards` component with a randomized scattered card layout.

Podcast
- `frontend/src/components/podcast.jsx` loads podcasts from backend via `getPodcasts()`.
- Supports search by title/author and tag filtering.
- Manages audio playback state, progress, duration, speed control, and interactive scroll animation.

Team
- `frontend/src/components/team.jsx` fetches team member data via `getTeam()`.
- Displays an interactive tile grid of members with animation, click-to-open profile detail, and preview overlay.
- Supports search input and tag-based filtering.
- Each tile can show profile links: GitHub, LinkedIn, email.
- Includes `portfolio/:username` navigation for team member profiles

Timeline
- `frontend/src/components/Timeline_Tree.jsx` and `Timeline_Dial.jsx` appear to render timeline visuals.
- Timeline page uses a tree-style component for event navigation.
- Likely presents timeline events in animated, interactive form.

Admin Panel
- `frontend/src/admin/components/Login.jsx` handles token-based login via `/api/login`.
- `ProtectedRoute` ensures dashboard access only after auth.
- `Dashboard.jsx` provides:
  - sidebar navigation for content types
  - search and filtering of current tab items
  - data fetch for blogs, videos, podcasts, team, timeline, and contact records
  - create/edit modal forms for blog/podcast/team/timeline/video management
  - delete actions for resources
  - logout functionality
- There are dedicated admin forms:
  - `BlogForm.jsx`
  - `PodcastForm.jsx`
  - `TeamForm.jsx`
  - `TimelineForm.jsx`
  - `VideoManager.jsx`

Frontend API Integration
- Base API URL: `http://localhost:8000`
- Auth token stored in `localStorage` and attached to requests by axios interceptor
- See the respective services in services.js

Future Implementation / Modification Ideas

Frontend Improvements
1. Add production API config.
   - Use env variables instead of hardcoded `http://localhost:8000`.
   - Use Vite env files for dev/prod base URLs.
2. Improve admin auth flow.
   - Use the same `/login` API path consistently and update the login client to use the backend base URL.
   - Add token expiry handling and refresh logic.
3. Add better error handling & UI feedback.
   - Replace `alert()` with toast notifications or modal messages.
   - Show API loading / error states consistently.
4. Add pagination and infinite scrolling.
   - For blog lists, podcast lists, team directory, and video cards.
5. Add user-facing blog list filters.
   - Search by title/tag, sort by date, show featured blogs.
6. Improve accessibility.
   - Ensure keyboard navigation, ARIA labels, contrast, and mobile friendliness.
7. Add comment author names, moderation view.
   - Allow visitors to add a name/email alongside comments.
   - Support admin comment approval/rejection.
8. Add offline/load fallback states.
   - Gracefully handle missing media, missing images, and backend timeouts.
9. Add client-side caching.
   - Cache video IDs, team data, and blog previews to reduce repeated API calls.

UX / Product Enhancements
1. Convert blog and podcast data to CMS-friendly structure.
2. Add a blog category/tag cloud and archive view.
3. Add a newsletter/contact success flow with redirect/thank-you page.
4. Add a proper admin user management page.
